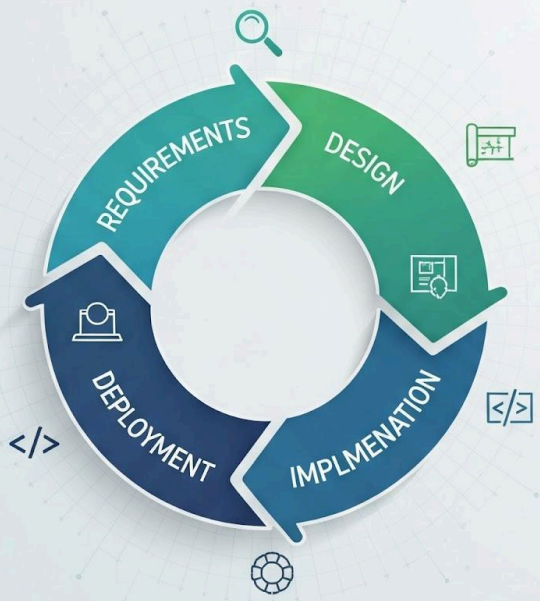


SOFTWARE DEVELOPMENT LIFE CYCLE

ÍNDICE

1. El Diálogo entre el Código y la Máquina 2
Analogía para entenderlo mejor: La Cocina Profesional 3
2. El Viaje del Código: De la Idea al Programa 3
La Hoja de Ruta: El Ciclo de Vida del Software 4
La Traducción Mágica: De Lenguaje Humano a Lenguaje Máquina 6
El Intérprete Universal: La Portabilidad y las Máquinas Virtuales 7
3. El Universo de los Lenguajes de Programación 7
4. El Arsenal del Desarrollador Moderno 12
5. El Arte de Construir en Equipo: Metodologías Ágiles 13
Scrum y Kanban: Enfoques Ágiles en Detalle 14
Scrum: El Marco para la Entrega Incremental 14
Kanban: La Gestión Visual del Flujo 15

¡Bienvenido/a al fascinante mundo del desarrollo de software! Estás a punto de embarcarte en un viaje que transformará tu manera de ver la tecnología. Un programa no es magia, es el resultado de un proceso creativo y lógico que convierte una idea en una herramienta funcional. Esta guía es tu mapa para ese viaje. Aquí desmitificaremos cada paso, desde cómo tu código "habla" con el metal del ordenador hasta cómo los equipos de élite colaboran para crear aplicaciones increíbles.

1. El Diálogo entre el Código y la Máquina

Antes de escribir nuestra primera línea de código, debemos entender dónde va a vivir y cómo se va a comportar. Un programa no existe en el vacío; es un invitado en un sistema informático y debe interactuar con sus componentes físicos (hardware) para poder funcionar.

- **El Procesador (CPU - Central Processing Unit):** Es el componente central que ejecuta las instrucciones de un programa. Funciona como el cerebro de la operación, leyendo comandos almacenados en la memoria uno por uno, decodificándolos y realizando las operaciones aritméticas, lógicas y de control que se le indican. Su rendimiento se mide en Gigahercios (GHz), que representa miles de millones de ciclos por segundo. Los procesadores modernos tienen múltiples **núcleos (cores)**, lo que les permite ejecutar varias tareas simultáneamente, mejorando drásticamente el rendimiento de las aplicaciones.
- **Memoria (RAM - Random Access Memory):** Es un almacenamiento temporal de alta velocidad donde se cargan tanto las instrucciones del programa como los datos con los que opera mientras está en ejecución. Cada dato o instrucción se almacena en una **dirección de memoria** única, como un casillero numerado, permitiendo a la CPU un acceso casi instantáneo. Es un recurso **volátil**, lo que significa que su contenido se pierde al apagar el sistema. La cantidad de RAM disponible limita cuántas aplicaciones y qué tan complejas pueden ser las que se ejecutan al mismo tiempo.
- **Los Periféricos y el Almacenamiento:** Los programas necesitan interactuar con el exterior.
    - **Dispositivos de Entrada/Salida:** Permiten la comunicación con el usuario y otros sistemas (teclado, ratón, pantalla, red).
    - **Dispositivos de Almacenamiento Permanente:** Sirven para guardar el software y los datos de forma persistente. La diferencia entre un **Disco** Duro tradicional **(HDD)** y

una **Unidad de Estado Sólido (SSD)** es crucial: los SSD son mucho más rápidos porque no tienen partes móviles, lo que reduce drásticamente los tiempos de carga del sistema operativo y de las aplicaciones.

- **El Sistema Operativo (SO):** Actúa como un intermediario fundamental y gestor de recursos entre el software y el hardware. Sus tareas principales incluyen:
  - **Gestión de procesos:** Decide qué programa utiliza la CPU y durante cuánto tiempo.
  - **Gestión de memoria:** Asigna porciones de RAM a cada aplicación y se asegura de que no interfieran entre ellas.
  - **Controladores de dispositivos (Drivers):** Proporciona una interfaz estandarizada para que el software pueda usar el hardware (impresoras, tarjetas gráficas) sin conocer los detalles técnicos de cada uno.

Analogía para entenderlo mejor: La Cocina Profesional

Para visualizar esta relación, imaginemos que tu **programa es un chef de alta cocina** y el **ordenador es una cocina de última generación**.

- El **Procesador (CPU)** serían las manos y el cerebro del chef, ejecutando cada paso de la receta (las instrucciones del programa) con precisión y rapidez. Un chef con más "núcleos" podría preparar varios platos a la vez.
- La **Memoria (RAM)** es la mesa de trabajo o encimera. Es donde el chef coloca los ingredientes (datos) y la receta que está preparando en ese momento para tener un acceso rápido y directo. Si se va la luz (se apaga el ordenador), todo lo que hay en la mesa se pierde.
- El **almacenamiento permanente** (disco duro/SSD) es la despensa o la nevera, donde se guardan todas las recetas y los ingredientes a largo plazo. Una nevera SSD permitiría encontrar los ingredientes mucho más rápido.
- El **Sistema Operativo** es el jefe de cocina. El chef no se preocupa de si la cocina es de gas o de inducción; simplemente le pide al jefe de cocina "enciende el fuego a 200 grados". El jefe de cocina se encarga de gestionar todas las herramientas y peticiones para que varios chefs puedan trabajar a la vez sin estorbarse.

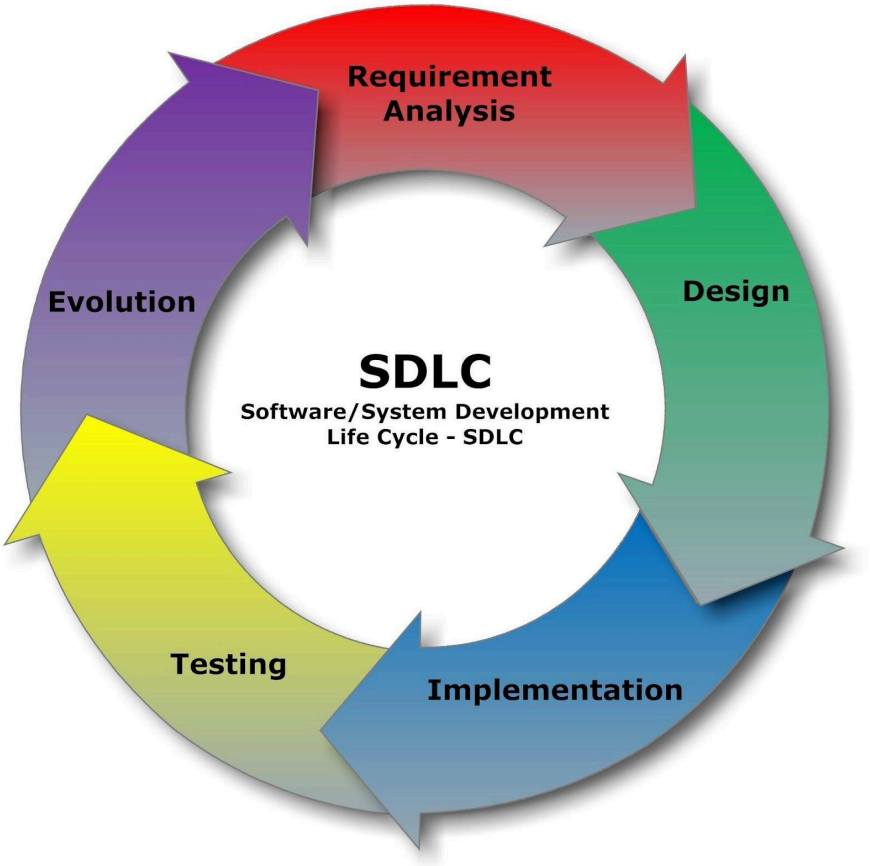
2. El Viaje del Código: De la Idea al Programa

Crear software es un proceso estructurado, un viaje que va desde una necesidad abstracta

hasta un producto tangible y funcional, conocido como el Ciclo de Vida del Desarrollo de Software (SDLC).

La Hoja de Ruta: El Ciclo de Vida del Software

Este ciclo define las fases por las que pasa un proyecto de software. Aunque existen muchos modelos, el modelo clásico en cascada ilustra bien las etapas secuenciales.



1. **Análisis de requisitos:** Se identifican y documentan las necesidades de los usuarios y los objetivos del negocio. Se utilizan técnicas como entrevistas, encuestas y la creación de **casos de uso** o **historias de usuario** para definir qué debe hacer el sistema. Es una fase crítica para el éxito del proyecto.
2. **Diseño:** Se define la arquitectura del software. Esto incluye el **Diseño de Alto Nivel** (la estructura general y cómo se comunican sus módulos) y el **Diseño** de Bajo **Nivel** (los detalles internos de cada módulo, como los algoritmos a usar). También se diseña la estructura de la base de datos (diagramas Entidad-Relación) y la experiencia de usuario

(UI/UX).

3. **Implementación:** Los desarrolladores escriben el código fuente del programa basándose en las especificaciones de la fase de diseño, utilizando los lenguajes y herramientas seleccionados.
4. **Pruebas (Testing):** Se verifica sistemáticamente que el software funciona según lo especificado. Se realizan distintos niveles de pruebas:
    - **Pruebas Unitarias:** Se prueba cada pequeña pieza de código de forma aislada.
    - **Pruebas de Integración:** Se comprueba que varias piezas funcionan correctamente juntas.
    - **Pruebas de Sistema:** Se prueba la aplicación completa para asegurar que cumple los requisitos.
    - **Pruebas de Aceptación (UAT):** El cliente o usuario final valida que el software hace lo que necesita.
5. **Despliegue:** El software se instala en el entorno de producción (por ejemplo, en servidores o tiendas de aplicaciones) para que los usuarios finales puedan empezar a utilizarlo.
6. **Mantenimiento:** Una vez desplegado, el software requiere un soporte continuo para corregir errores que surjan con el uso, adaptarse a nuevos requisitos o sistemas operativos y añadir mejoras funcionales.

La implementación de un ciclo de vida para el desarrollo de software, ya sea mediante un enfoque secuencial como el modelo en cascada o a través de metodologías ágiles e iterativas, va más allá de la simple formalidad. Es un pilar fundamental en la creación de software profesional. Cada etapa de este ciclo, desde la definición y análisis de requisitos hasta el mantenimiento y soporte, influye de manera crítica en la **calidad** del producto final. Omitir o subestimar cualquiera de estas fases lleva inevitablemente a la aparición de defectos, la necesidad de rehacer trabajos que cuestan mucho y, en última instancia, al riesgo de fracaso del proyecto.

Además, un proceso de desarrollo bien estructurado asegura la **eficiencia** en la gestión de recursos y la optimización de los plazos. Una planificación y un diseño cuidadosos y anticipados permiten minimizar los problemas inesperados y maximizar la efectividad del

esfuerzo de desarrollo. Las pruebas sistemáticas, por su parte, garantizan que el software no solo cumpla con su propósito funcional, sino que lo haga de manera robusta y fiable, sin incidencias inesperadas. Finalmente, el software concebido bajo un ciclo de vida adecuadamente ejecutado posee una intrínseca **mantenibilidad**. La documentación completa, un diseño coherente y un código estructurado facilitan significativamente las futuras actualizaciones, la corrección de errores y la adaptación a nuevos requerimientos, extendiendo así la vida útil y el valor estratégico del sistema. En resumen, seguir de forma disciplinada el ciclo de vida del software es indispensable para transformar una idea inicial en una solución tecnológica duradera, de alto rendimiento y alineada con los estándares profesionales.

La Traducción Mágica: De Lenguaje Humano a Lenguaje Máquina

El procesador de cualquier sistema informático opera con **lenguaje máquina, compuesto por código binario**. Las **instrucciones de programación, escritas en lenguajes de alto nivel como Python o Java, deben convertirse a este lenguaje**.

Este proceso de conversión, **compilación o interpretación, es crucial**. La **compilación traduce el código fuente completo antes de la ejecución**, mientras que la interpretación traduce y ejecuta línea por línea en tiempo real. La elección depende del lenguaje y los requisitos.

En resumen, **este proceso une la lógica humana con la lógica de la máquina, transformando instrucciones complejas en operaciones binarias que el procesador puede ejecutar**.

1. **Código Fuente:** Es el conjunto de instrucciones escritas por un programador en un lenguaje de alto nivel (Java, Python, C#), que es legible para los humanos.
2. **Código Objeto:** Es el resultado de procesar el código fuente con un **compilador**. Este programa realiza un análisis léxico y sintáctico del código fuente y lo traduce a lenguaje máquina o a un formato intermedio. Este código aún no es un programa autónomo, ya que puede requerir piezas externas de código (librerías).

3. **Código Ejecutable:** Es el archivo final, generado por un **enlazador** (*linker*), que une uno o más archivos de código objeto con el código de las librerías necesarias. Resuelve las referencias entre los distintos ficheros y crea el programa final, listo para ser ejecutado por el sistema operativo.

El Intérprete Universal: La Portabilidad y las Máquinas Virtuales

Para evitar tener que reescribir y recompilar un programa para cada sistema operativo, algunos lenguajes utilizan un enfoque híbrido. El código fuente se compila a un **código intermedio** (como *Bytecode* en Java o CIL en .NET). Este código es ejecutado por una **Máquina Virtual (VM)**, un software que actúa como un ordenador simulado. La VM utiliza un compilador **Just-In-Time (JIT)** que traduce el código intermedio a código máquina nativo en el momento de la ejecución, optimizando el rendimiento.

Analogía para entenderlo mejor: La Receta de Cocina Universal

- En lugar de escribir una receta de paella en español, francés y chino, la escribes en un **lenguaje universal de recetas (código intermedio)**.
- La **Máquina Virtual** es un chef local. Para hacer la paella en España, un chef español (la JVM para Windows) lee la receta universal y usa los ingredientes y utensilios locales. En Francia, un chef francés (la JVM para macOS) hará lo mismo con sus propios utensilios. La receta es la misma, pero el chef local la adapta al entorno sobre la marcha.

3. El Universo de los Lenguajes de Programación

Los lenguajes de programación son herramientas fundamentales en el desarrollo de software, y su diversidad permite abordar una amplia gama de problemas y requisitos. Para comprender mejor esta diversidad, se pueden clasificar según varios criterios clave:

Por Nivel de Abstracción

Esta clasificación se refiere a cuán cerca o lejos está el lenguaje del hardware de la

computadora.

- **Bajo Nivel:** Estos lenguajes están muy cerca de la arquitectura de la máquina. Ofrecen un control extremadamente preciso sobre el hardware, lo que los hace ideales para tareas donde el rendimiento es crítico o donde se necesita interactuar directamente con componentes específicos del sistema. Sin embargo, su complejidad es considerable, requiriendo un conocimiento profundo de la arquitectura del procesador.
    - **Ejemplo:** El **Ensamblador** es un claro ejemplo. Cada instrucción en ensamblador se corresponde casi directamente con una instrucción del procesador. Es utilizado en sistemas embebidos, controladores de dispositivos y optimización de rendimiento crítico.
- **Alto Nivel:** Estos lenguajes se diseñan para ser más cercanos al lenguaje humano y abstractos de la complejidad subyacente del hardware. Facilitan la escritura de código, reducen el tiempo de desarrollo y son más fáciles de aprender y mantener.
    - **Ejemplos:**
        - **Python:** Conocido por su legibilidad y versatilidad, es ampliamente utilizado en desarrollo web, ciencia de datos, inteligencia artificial y automatización.
        - **Java:** Un lenguaje robusto y orientado a objetos, popular en aplicaciones empresariales, desarrollo móvil (Android) y sistemas distribuidos. Su lema "escribir una vez, ejecutar en cualquier lugar" resalta su portabilidad.
        - **C# (C Sharp):** Desarrollado por Microsoft, es un lenguaje orientado a objetos utilizado principalmente en el ecosistema .NET para aplicaciones de escritorio, web, juegos (Unity) y la nube.

Por Paradigma de Programación

Un paradigma de programación es un estilo fundamental de programación, una forma de clasificar los lenguajes de programación según sus características.

- **Imperativo:** En este paradigma, el programador describe una secuencia explícita de instrucciones o comandos que la computadora debe ejecutar para cambiar el estado del programa. Se centra en el *cómo* se realiza la computación.
    - **Ejemplo: C** es un lenguaje imperativo por excelencia. Los programas en C se

construyen a partir de una serie de funciones que manipulan datos y controlan el flujo de ejecución mediante bucles y condicionales.

- **Orientado a Objetos (OOP):** Este paradigma modela el programa como una colección de "objetos" que encapsulan datos y el comportamiento relacionado con esos datos. Los objetos interactúan entre sí a través de mensajes. Sus pilares fundamentales son:
    - **Encapsulación:** Agrupa datos y métodos que operan sobre esos datos en una única unidad (el objeto), ocultando los detalles internos y exponiendo solo una interfaz controlada.
    - **Herencia:** Permite que una clase (clase hija) adquiera propiedades y comportamientos de otra clase (clase padre), promoviendo la reutilización de código y la creación de jerarquías.
    - **Polimorfismo:** Permite que objetos de diferentes clases respondan al mismo mensaje de manera diferente, adaptándose a su propio tipo. Facilita la flexibilidad y extensibilidad del código.
    - **Abstracción:** Se enfoca en los aspectos esenciales de un objeto, ocultando la complejidad subyacente y presentando solo la información relevante.
    - **Ejemplos:**
        - **Java:** Fuerte soporte para OOP, con un modelo de objetos claro y robusto.
        - **C#:** También con un fuerte enfoque en OOP, integrado con el framework .NET.
- **Declarativo:** En contraste con el imperativo, el paradigma declarativo se enfoca en describir *qué* se quiere lograr, en lugar de especificar *cómo* se deben ejecutar las operaciones. El sistema se encarga de determinar la secuencia de pasos para alcanzar el objetivo. Un subtipo importante es la Programación Funcional.
    - **Programación Funcional:** Un sub-paradigma del declarativo que trata la computación como la evaluación de funciones matemáticas y evita el estado mutable y los datos cambiantes. Se enfoca en funciones puras (que no tienen efectos secundarios) e inmutabilidad.
    - **Ejemplos:**
        - **SQL (Structured Query Language):** Se utiliza para gestionar y manipular bases de datos relacionales. Declarativamente, se especifica qué datos se desean obtener o modificar, sin detallar cómo el motor de la

base de datos debe realizar esa operación.

- **Haskell:** Un lenguaje puramente funcional, conocido por su tipado estático fuerte y su énfasis en la inmutabilidad y la composición de funciones.

Por Forma de Ejecución

Esta clasificación se refiere a cómo el código fuente de un programa se convierte y ejecuta en instrucciones que la máquina puede entender.

- **Compilados:** Estos lenguajes requieren un "compilador" que traduce el código fuente completo a un archivo ejecutable (código máquina) antes de que el programa pueda ser ejecutado. Una vez compilado, el ejecutable puede correr de forma independiente y generalmente es más rápido.
  - **Ventajas:** Mayor rendimiento, detección temprana de errores (en tiempo de compilación).
  - **Desventajas:** Proceso de compilación puede ser lento, menos portabilidad si el ejecutable no se compila para diferentes arquitecturas.
  - **Ejemplos:**
    - **C++:** Utilizado en sistemas operativos, juegos, software de alto rendimiento.
    - **C:** Similar a C++, fundamental en sistemas operativos y desarrollo de firmware.
- **Interpretados:** En este modelo, un "intérprete" lee y ejecuta el código fuente línea por línea en tiempo de ejecución, sin una fase de compilación previa a un ejecutable.
  - **Ventajas:** Mayor flexibilidad, mayor portabilidad (el mismo código fuente puede ejecutarse en diferentes plataformas con el intérprete adecuado), ciclo de desarrollo más rápido (sin espera de compilación).
  - **Desventajas:** Generalmente más lentos en ejecución que los compilados, ya que el intérprete debe procesar el código en cada ejecución.
  - **Ejemplos:**
    - **Python:** Su flexibilidad y facilidad de uso lo hacen popular para scripting y desarrollo rápido.
    - **JavaScript:** Esencial para el desarrollo web front-end, ejecutándose

directamente en los navegadores.

- **Híbridos:** Estos lenguajes combinan aspectos de la compilación y la interpretación. Generalmente, el código fuente se compila a un formato intermedio (bytecode) que luego es ejecutado por una máquina virtual (VM) en tiempo de ejecución.
    - **Ventajas:** Mayor portabilidad que los lenguajes puramente compilados (el bytecode es independiente de la plataforma), mejor rendimiento que los lenguajes puramente interpretados gracias a la optimización de la VM.
    - **Desventajas:** Requiere una VM instalada en el sistema de destino.
    - **Ejemplos:**
        - **Java:** El código fuente Java se compila a bytecode que se ejecuta en la Java Virtual Machine (JVM).
        - **C#:** El código C# se compila a Common Intermediate Language (CIL), que luego es ejecutado por el Common Language Runtime (CLR) de .NET.

Por Sistema de Tipos

El sistema de tipos de un lenguaje de programación define cómo se manejan los tipos de datos y cuándo se realizan las comprobaciones de tipo.

- **Tipado Estático:** En lenguajes con tipado estático, los tipos de las variables se comprueban en tiempo de compilación. Esto significa que el tipo de cada variable debe ser conocido antes de que el programa se ejecute.
    - **Ventajas:** Ayuda a detectar una amplia gama de errores de tipo en una etapa temprana (antes de la ejecución), lo que conduce a un código más robusto y menos propenso a errores en producción. Permite optimizaciones de rendimiento por parte del compilador.
    - **Desventajas:** Menos flexibilidad, requiere que el programador declare explícitamente los tipos o que el compilador los infiera.
    - **Ejemplos:**
        - **Java:** Todas las variables tienen un tipo definido que se verifica en compilación.
        - **C++:** Al igual que Java, los tipos son fundamentales y se verifican en la compilación.

- **Tipado Dinámico:** En lenguajes con tipado dinámico, los tipos de las variables se comprueban en tiempo de ejecución. Esto significa que el tipo de una variable puede cambiar durante la ejecución del programa.
    - **Ventajas:** Ofrece mayor flexibilidad y agilidad en el desarrollo, lo que puede acelerar el prototipado y la escritura de código. Menos código boilerplate relacionado con la declaración de tipos.
    - **Desventajas:** Los errores de tipo pueden no ser detectados hasta que el programa se ejecuta y la línea de código problemática es alcanzada, lo que puede llevar a errores en tiempo de ejecución más difíciles de depurar.
    - **Ejemplos:**
        - **Python:** Las variables no tienen un tipo fijo; el tipo se asocia al valor en tiempo de ejecución.
        - **JavaScript:** Similar a Python, las variables pueden contener valores de diferentes tipos a lo largo de la ejecución.

Esta clasificación exhaustiva ayuda a los desarrolladores a elegir el lenguaje de programación más adecuado para un proyecto específico, considerando los requisitos de rendimiento, portabilidad, facilidad de desarrollo y mantenimiento, y la naturaleza del problema a resolver.

4. El Arsenal del Desarrollador Moderno

El desarrollo de software profesional se apoya en un ecosistema de herramientas diseñadas para maximizar la productividad y la calidad, abarcando desde la creación de código hasta la entrega final del producto.

- **IDE (Entorno de Desarrollo Integrado):** Una aplicación centralizada que proporciona todas las herramientas necesarias para la codificación. Incluye un editor de código avanzado con características como autocompletado inteligente, resaltado de sintaxis, refactorización (reestructuración del código sin cambiar su comportamiento), y navegación de código. También integra un depurador para identificar y corregir errores, y sistemas de automatización de tareas. Ejemplos populares son Visual Studio Code (ligero y extensible) e IntelliJ IDEA (potente y con muchas funciones para Java y otros lenguajes).

- **Sistemas de Control de Versiones (VCS):** Software esencial que registra cada cambio realizado en un proyecto, permitiendo rastrear el historial, revertir a versiones anteriores y gestionar múltiples líneas de desarrollo. Git es el estándar de la industria y permite a los equipos trabajar en "ramas" (líneas de desarrollo paralelas) para implementar nuevas características o corregir errores de forma aislada. Una vez que el trabajo en una rama está completo y probado, se puede fusionar con la rama principal, facilitando la colaboración y la experimentación sin riesgos de afectar el código base principal.
- **Herramientas de Construcción (Build Tools):** Automatizan el proceso de transformar el código fuente en una aplicación ejecutable. Esto incluye compilar el código (convertirlo de un lenguaje de alto nivel a uno que la máquina pueda entender), gestionar dependencias (librerías externas o módulos que tu proyecto necesita para funcionar correctamente y que son descargadas automáticamente), ejecutar pruebas automatizadas y empaquetar la aplicación en un formato distribuible. Herramientas como Maven y Gradle son fundamentales en el ecosistema Java para gestionar el ciclo de vida de la construcción del proyecto de manera eficiente.

5. El Arte de Construir en Equipo: Metodologías Ágiles

Frente a los modelos tradicionales y rígidos (en cascada), las metodologías ágiles proponen un enfoque basado en la flexibilidad, la colaboración y la entrega de valor incremental y continuo.

Analogía para entenderlo mejor: Construir un Puente vs. Cultivar un **Jardín**

- El **desarrollo tradicional (cascada)** es como **construir un puente**: se requiere una planificación exhaustiva al inicio, y una vez que la construcción comienza, el plan se sigue de forma rígida. Los cambios son muy costosos.
- El **desarrollo ágil** es como **cultivar un jardín**: se empieza por una pequeña parcela (**sprint**), se obtiene un resultado visible (valor temprano), se recoge la opinión del "dueño" del jardín (cliente), y con esa retroalimentación se planifica la siguiente parcela.

Marcos de Trabajo Ágiles Populares:

- Scrum: Se basa en ciclos de trabajo de duración fija llamados sprints. Define roles

(Product Owner, Scrum Master, Equipo) y eventos (Daily, Review, Retrospective) para fomentar un ritmo de trabajo sostenido y una mejora continua.

- Kanban: Es un método visual para gestionar el flujo de trabajo. Utiliza un tablero para representar las fases del proceso y limita el trabajo en curso (WIP) para evitar cuellos de botella y optimizar el tiempo de entrega.

Scrum y Kanban: Enfoques Ágiles en Detalle

Si bien el desarrollo ágil es una filosofía, existen marcos de trabajo específicos que guían su implementación. Los más populares son Scrum y Kanban, cada uno con sus propias características distintivas.

Scrum: El Marco para la Entrega Incremental

Scrum es un marco de trabajo iterativo e incremental que se enfoca en entregar valor de forma regular y predecible. Se basa en ciclos de trabajo cortos y de duración fija, conocidos como "sprints", que suelen durar de una a cuatro semanas. Durante cada sprint, el equipo trabaja en un conjunto de elementos seleccionados del "Product Backlog" (una lista priorizada de características y requisitos).

Roles Clave:

- **Product Owner:** Representa los intereses del cliente y del negocio. Es responsable de maximizar el valor del producto y de gestionar el Product Backlog.
- **Scrum Master:** Facilita el proceso Scrum, elimina impedimentos y ayuda al equipo a seguir las prácticas ágiles. No es un gerente, sino un líder de servicio.
- **Equipo de Desarrollo:** Un grupo autoorganizado y multifuncional de profesionales que realiza el trabajo de crear el incremento del producto.

Eventos Clave:

- **Sprint Planning:** El equipo planifica el trabajo a realizar en el próximo sprint.

- **Daily Scrum (Stand-up):** Una reunión diaria corta para sincronizar actividades y planificar el trabajo del día.
- **Sprint Review:** El equipo presenta el incremento de producto completado a los stakeholders para obtener retroalimentación.
- **Sprint Retrospective:** El equipo reflexiona sobre el sprint que acaba de terminar y planifica mejoras para el siguiente.

Kanban: La Gestión Visual del Flujo

Kanban es un método para gestionar y mejorar el flujo de trabajo, visualizando el proceso y limitando el trabajo en curso (WIP - Work In Progress). Su principal herramienta es el tablero Kanban, que representa las diferentes etapas del proceso de desarrollo.

Principios Clave:

- **Visualizar el flujo de trabajo:** Las tareas se representan como tarjetas en un tablero, moviéndose a través de columnas que simbolizan las etapas del proceso (Por Hacer, En Progreso, Hecho).
- **Limitar el trabajo en curso (WIP):** Se establece un número máximo de tareas que pueden estar en cada etapa simultáneamente. Esto ayuda a identificar cuellos de botella y a mantener un flujo constante.
- **Gestionar el flujo:** El enfoque está en mover las tareas de forma fluida a través del tablero, buscando optimizar el tiempo que tarda una tarea en completarse.
- **Hacer las políticas explícitas:** Las reglas para mover las tareas y los criterios para considerarlas "hechas" son claros y visibles para todos.

Ventajas:

- **Flexibilidad:** Ideal para proyectos donde los requisitos cambian frecuentemente o donde el flujo de trabajo es más impredecible.
- **Reducción de cuellos de botella:** Al limitar el WIP, se fomenta que el equipo se enfoque en terminar el trabajo antes de empezar nuevas tareas.
- **Transparencia:** El tablero Kanban ofrece una visión clara del estado del proyecto en

todo momento.

Scrum y Kanban son metodologías ágiles que comparten el objetivo de optimizar la eficiencia y la adaptabilidad en el desarrollo de software, aunque difieren en su enfoque. Scrum ofrece una estructura más definida en cuanto a roles y eventos, lo que lo hace idóneo para equipos que buscan una cadencia de entrega regular y previsible. Por otro lado, Kanban se integra con mayor facilidad en los flujos de trabajo existentes y prioriza la mejora continua mediante una gestión visual.

-